

RESEARCH

Open Access



Intrusion detection latency: the neglected metric

Sandhyarani Dash^{1*} and John M. Acken¹

Abstract

Accuracy remains the dominant evaluation metric in Intrusion Detection System (IDS) research, yet an IDS that detects attacks too late is functionally equivalent to one that fails—particularly in Internet of Things (IoT) environments. In operational settings, the timing of detection shapes both the scope of adversarial activity and the feasibility of effective response. To the best of our knowledge, latency (the speed at which intrusions are identified) has received no systematic attention. Our analysis of published IDS papers reveals that latency is defined inconsistently—often referring to inference, communication, computation time, or combinations of these—leading to incomparable performance claims. To close this gap, we formally define end-to-end latency as the sum of three measurable components: *Attacker-Controlled Latency* (ACL), *IDS-Determined Latency* (IDL), and *Post-Detection Latency* (PDL). The framework further decomposes IDL into environment-specific sub-components, linking network topology and detection methodology to overall responsiveness. The framework provides a systematic mechanism to parameterize and vary these components across deployment environments (edge, cloud, federated learning). Using a Cyber-Physical dataset, we demonstrate the applicability of our latency framework to an existing dataset. By providing a mechanism to quantify detection timeliness, the proposed framework enables analysts to estimate attacker dwell time, benchmark real-time responsiveness, and standardize latency reporting across IDS studies.

Keywords Intrusion detection systems, Latency, Cyber-physical systems, Network security, Evaluation metrics, Attack detection latency

Introduction

The frequency and sophistication of cyberattacks have escalated with the rise of Generative AI (GenAI), which enables automated cyber-attacks such as phishing, impersonation, and payload creation with minimal effort (Usman et al. 2024). Accuracy remains the dominant benchmark in Intrusion Detection System (IDS) research, yet a detection that occurs too late is effectively a failure, as timing ultimately determines the extent of damage an attacker can inflict. Despite this, most IDS studies continue to prioritize accuracy while neglecting latency—often using the term inconsistently or without formal

definition—which obscures comparability and real-time relevance. This gap is critical for Cyber-Physical Systems (CPS) and IoT environments, where even millisecond-scale delays across power grids, autonomous vehicles, or medical devices can trigger cascading failures, underscoring the need for standardized latency definition for fair comparison across IDS designs and more informed architectural decisions for latency reduction. These challenges motivate the following research questions:

1. **RQ1:** How is intrusion detection latency defined and used across existing IDS literature?
2. **RQ2:** Do current IDS datasets provide latency-complete temporal annotations necessary for accurate latency calculation?
3. **RQ3:** How can end-to-end IDS latency be formally decomposed into measurable components?

*Correspondence:

Sandhyarani Dash
dashes@pdx.edu

¹ Department of Electrical and Computer Engineering, Portland State University, Portland, OR 97201, USA

4. RQ4: How do we measure latency?

Grounded in the research questions, the contributions of this work collectively elevate latency to a first-class metric in IDS research:

1. *Systematic Evidence*: A structured review of 90 IDS papers revealing inconsistent or undefined use of the term *latency*. Addressed in Sect. 2.1.
2. *Root Cause Analysis*: A dataset landscape analysis explaining why existing corpora are *latency-partial* and outlining requirements for *latency-complete* datasets and multi-metric evaluation. Addressed in Sect. 2.2.
3. *Formal Framework and Case Studies*: A mathematical decomposition of end-to-end latency into three additive components—Attacker-Controlled Latency (ACL), IDS-Determined Latency (IDL), and Post-Detection Latency (PDL)—with case studies illustrating latency optimization across environments, and derivation of Attack Detection Latency (ADL) as the measurable operational subset for empirical evaluation. Addressed in Sect. 3.
4. *Empirical Demonstration*: Application of the proposed framework to compute Attack Detection Latency (ADL) on the ROSPaCe (Puccetti et al. 2024) CPS dataset. Addressed in Sect. 4.

Related work

What is published about latency in IDS literature?

To investigate the above research questions, we conducted a structured analysis of IDS papers from IEEE using the search criteria summarized in Table 1. As shown in the same table, the search returned a total of 95 papers (46 + 28 + 21), of which 15 were duplicates across results, yielding 80 unique IEEE papers reviewed. Each paper was examined for how latency was defined, measured, or discussed. While our structured analysis primarily draws from IEEE Xplore, we also examined

a total of five representative ACM (Borgioli et al. 2023; Asulba et al. 2025; Yagiz et al. 2024; Zhang et al. 2025) and USENIX (Xu et al. 2024) papers that explicitly quantify inference or detection latency in real-time or embedded settings. The additional papers were selected based on topical relevance rather than keyword retrieval.

In addition, we examined five IDS studies that employ traditional, non-learning-based detection schemes, including rule-based, signature-based, distance-based, protocol-based, and threshold-based techniques. To identify relevant work, we queried the IEEE Xplore digital library using the keywords Network Intrusion Detection System AND “Time Overhead” as shown in Table 1, restricting the search to journal publications. This search yielded 22 papers published between 2011 and 2025. After screening all abstracts, we selected five studies that explicitly report time overhead while relying exclusively on non-learning based detection mechanisms.

Across these works, time overhead is consistently defined as the computational cost of making a detection decision under the assumption that all required inputs are already available, rather than as the total time a malicious packet remains undetected while traversing the processing pipeline. For example, Wu et al. (2016) evaluate runtime overhead at the decision layer after sensor data collection and formatting have been completed. Similarly, Mohammadi et al. (2017) assess detection time in an SDN-based countermeasure by measuring controller-side response latency, assuming timely availability of flow statistics. In vehicular IDS, Liang et al. (2020) focus on the processing cost of feature extraction and decision-making within the IDS, while treating packet capture and buffering as external to the reported time overhead.

Recent wireless IDS work by Thankappan et al. (2024) further illustrates this abstraction. Although their system implements a complete IDS pipeline—comprising passive packet capture in monitor mode, buffering via a sliding probe interval, frame parsing and event counting, hand-crafted feature computation, and threshold-based decision logic executed at the end of each probe

Table 1 Summary of IEEE search results by keyword combination

IEEE keywords	Journal/conference	Year	Paper count
Latency AND Intrusion Detection System AND Accuracy AND Machine Learning	Journal Only	2007–2025	48 papers retrieved; 2 retracted; 46 papers retained for review
Latency AND Intrusion Detection System	Journal Only	2022–2025	210 papers retrieved; 28 highly cited (≥ 40 citations) retained for review
Edge Computing AND Security AND Machine Learning AND Synthetic Data	Journal and Conference	2022–2024	37 total; 31 relevant to IDS but 21 focused on attack detection which were retained for review
Network Intrusion Detection System AND “Time Overhead”	Journal Only	2011–2025	22 papers; Abstracts were screened and 5 rule-based and signature-based IDS studies were reviewed in detail

interval—the reported detection time corresponds solely to the fixed 60 s observation window. No breakdown is provided for the time incurred by data collection, data processing, or feature extraction. A similar assumption is made explicit in Paradise, a distributed provenance-based IDS proposed by Wu et al. (2023), where cluster detection time is reported only after extensive preprocessing. The authors note that “after an average of 86.86 s of preprocessing steps, we finally get 3.3 MB processed feature vector data that needs to be distributed for detection,” and define time overhead as “the test time overhead is the average of running the intrusion detection method six times.”

Taken together, these studies adopt a consistent abstraction in which time overhead corresponds to decision-time cost—analogue to inference time in learning-based IDS—while excluding data collection, buffering, data processing, and feature extraction stages. As a result, the reported time overhead metrics systematically underrepresent the true attack detection latency experienced in operational deployments.

Table 2 presents the eight categories into which 90 papers were classified, based on the following four criteria:

1. Latency is neither defined nor measured.
2. Latency is explicitly defined as:
 - Inference Time
 - Compute Time
 - Communication Time
3. Latency is defined as combination of inference, compute and communication time.
4. Latency is defined as time overhead.

Overall in the structured analysis of 90 IDS papers, we observe that the term *latency* is used inconsistently across the literature, often referring to partial aspects such as inference, communication, or computation time, or some combination thereof. None of the reviewed works presents a unified framework for quantifying *end-to-end latency*—defined as the total time from the onset of an attack to its mitigation. To address this gap, we formalize latency as a measurable, system-level property composed of three distinct yet additive components. This formalization clarifies what constitutes timely detection and enables objective comparison across IDS architectures and datasets, while guiding architectural decisions through end-to-end latency analysis rather than isolated optimization of computation, communication, or inference delays.

Assessing latency completeness in IDS datasets

Accurate latency measurement requires datasets that preserve packet-level granularity, continuous timestamps, and interleaved benign and attack traffic. However, most widely used IDS corpora lack even locally continuous timestamps and instead aggregate traffic into flow-level summaries, thereby breaking the temporal continuity essential for precise latency quantification. In addition, many datasets place attacks in contiguous blocks—often toward the end of the dataset—which complicates data splitting for Machine Learning (ML)/Deep Learning (DL) models unless samples are shuffled, thereby breaking temporal integrity. These design choices—introduced as early as DARPA'98 and perpetuated in datasets such as NSL-KDD, UNSW-NB15, CICIDS2017, and several IoT datasets—obscuring the fine-grained temporal

Table 2 Categorization of IDS papers by interpretation of latency. Full source lists for each category are provided in Appendix

Category	Interpretation of “latency”	No. of papers	Source list (Appendix)
Do not discuss latency	Latency not discussed; accuracy is primary metric or latency is mentioned but neither defined nor measured	24	[A]
Inference time	Time required for classifying traffic into attack/normal classes	18	[B]
Compute time	Time required for data preprocessing, feature extraction, feature engineering before classification	4	[C]
Communication time	Network latency, routing delay, packet delivery delay, QoS delay, or communication overhead in federated/distributed learning	29	[D]
Inference + Compute time	Joint delay of data preprocessing/feature extraction and model inference	3	[E]
Compute + Communication time	Combined delay of preprocessing/feature extraction and data transfer between edge and cloud layers	3	[F]
Compute + Communication + Inference time	delay for preprocessing/feature steps, packet transmission/network delay, and classification	5	[G]
Time overhead	time to apply rules/signatures once inputs are ready	5	[H]

sequencing needed to identify both the onset of an attack and the moment of its detection. As a result, although such datasets are well suited for accuracy benchmarking, they are fundamentally inadequate for evaluating IDS latency under realistic, mixed-traffic conditions. Table 3 summarizes how the design of commonly used IDS datasets has evolved and its implications for latency analysis.

The temporal formats in Table 3 are classified as follows:

1. *Non-continuous (flow)*—Flow-based data without per-packet timestamps.
2. *Non-continuous (packet)*—Packet-level data where timestamps are continuous only within each PCAP file. Timestamps reset across PCAP files, and each file represents a single scenario (benign or attack). The absence of natural benign–attack transitions can create misleading temporal patterns for sequence models such as Long Short-Term Memory (LSTM) networks.
3. *Continuous (packet)*—Packet-level data with globally continuous timestamps and interleaved benign and attack traffic within one timeline. This format is ideal for latency measurement.

In contrast, the ROSPaCe dataset (Puccetti et al. 2024) preserves packet-level structure with globally continuous timestamps and interleaved benign–attack traffic, enabling precise per-packet ordering across network, OS, and ROS2 layers. This design labels attack start and attack end with its associated timestamps required for computing Attack Detection Latency (ADL) under our framework. While ROSPaCe marks a major step toward latency-aware IDS evaluation, it remains partially annotated, lacking all IDL components. Thus, it serves as the first practical foundation for time-series latency analysis but not yet a *latency-complete* dataset. The next section formalizes latency as composite of-ACL, IDL, and PDL—and presents case studies highlighting which IDL stage most constrains system responsiveness, followed by the derivation ADL.

Formal definition of latency

Definition of start of an attack

A critical step in formalizing latency is defining when an attack begins. In cybersecurity literature, this point aligns with *reconnaissance*—the initial phase of the intrusion kill chain where adversaries gather information about targets through active probing or passive observation.

Table 3 Comparison of IDS datasets: evolution, temporal structure, and limitations

Dataset	Year	Size	Attack categories	Temporal format	Key advancement	Limitations
DARPA'98	1998	4 GB PCAPs (7 wks train, 2 wks test)	DoS, R2L, U2R, Probing	Non-continuous (packet)	First large-scale benchmark with labeled traffic	Synthetic traffic, unrealistic user behavior, exploitable artifacts
KDD Cup'99	1999	4.9M train, 311K test records	Same four DARPA categories (41 features)	Non-continuous (flow)	Made IDS data widely accessible to ML community	Redundancy, imbalance, inherits DARPA flaws
NSL-KDD	2009	126K train, 22K test	DoS, R2L, U2R, Probing	Non-continuous (flow)	Removed redundancy, balanced classes	Still synthetic, outdated traffic
UNSW-NB15	2015	2.5M records, 100 GB PCAPs	9 families (Fuzzers, DoS, Recon, etc.)	Non-continuous (flow)	Modern traffic mix, realistic attacks, 49 features	Flow CSVs dominate use, testbed-based
CICIDS2017	2017	3 M flows, 100 GB PCAPs	DoS/DDoS, Brute Force, Botnet, Web, Infiltration, Port Scan	Non-continuous (flow)	Realistic benign profiles, wide attack diversity	Packet timing lost in feature extraction
N-BaloT	2018	70 GB PCAPs (100 M packets)	Botnet malware (Mirai, Bashlite)	Non-continuous (packet)	First IoT malware dataset with real devices	Narrow scope, device-specific
BoT-IoT	2019	70 GB, billions of records	DDoS, DoS, Recon, Info Theft	Non-continuous (flow)	Large-scale IoT botnet simulation	Flow CSV dominates, severe imbalance
IoT-23	2023	66 GB PCAPs (16 M packets)	Malware, Cryptomining, C2	Non-continuous (packet)	First large open IoT dataset with labeled PCAPs	Limited devices, imbalance (few attacks vs many benign)
CICIoT2023	2023	548 GB, 105 devices, 33 attacks	DDoS, DoS, Botnets, Recon, Exfiltration	Non-continuous (flow)	Largest IoT dataset, multi-environment coverage	Flow CSVs dominate, imbalance, scalability issues
ROSPaCe	2024	30 M records, 38 GB (full), 14.5 GB (reduced)	6 attacks (2 discovery, 2 DoS, 2 ROS2-specific attacks)	Continuous (packet)	First to interlace benign and attack traffic at packet-level time granularity	Limited to 6 attacks, ROS2-focused scope

Hutchins et al. Hutchins et al. (2011), Mandiant MITRE Corporation (2025) and MITRE ATT&CK Mandiant (2020) describe it as a preparatory phase aimed at identifying exploitable vulnerabilities for later intrusion stages. Although not all attacks include this phase, reconnaissance traffic—such as scanning or probing—marks the onset of hostile activity, a view supported by intrusion detection studies that treat it as malicious even when no exploitation follows Sommer and Paxson (2010) and Lippmann et al. (2000).

Based on this perspective, we define the **start of an attack** as follows:

The point in time when an adversary first transmits malicious or suspicious content—malicious traffic, exploit attempts, or probing payloads—toward the target system. Importantly, this definition does not depend on whether the payload ultimately succeeds or becomes active. The presence of hostile communication itself signals the initiation of the attack.

This framing ensures that latency is consistently measured from the moment adversarial traffic first interacts with the monitored environment.

Dimensions that influence latency

Latency in intrusion detection is influenced by two orthogonal dimensions: *detection methodology* and *deployment environment*.

1. Detection Methodology: IDS can be broadly categorized based on how they detect malicious activity:

- *Signature-based IDS (SIDS)* rely on pattern matching against known attack signatures (e.g., rule sets, regular expressions). Because detection involves deterministic matching, these systems are fast and generally introduce minimal latency.
- *Anomaly-based IDS (AIDS)* detect deviations from learned or predefined models of normal behavior. These often involve feature extraction, statistical analysis, or ML inference—each of which contributes to overall latency.
- *Hybrid IDS* combine signature and anomaly-based components to improve detection coverage and speed of detection. Latency in such systems varies depending on the interplay between their components.
- *Host-based IDS (HIDS)* monitor internal host activities such as logs, file integrity, or system calls. These checks are often performed at fixed intervals or triggered by specific events. In such cases, *latency is not formally defined*, since detection

delay depends on the monitoring cadence rather than real-time analysis.

- *Network-based IDS (NIDS)* analyze network traffic at the packet or flow level. Because they operate on continuously arriving data, they allow detection latency to be measured as a well-defined metric.

The IDL part of latency framework developed in this work is designed specifically for Machine-Learning based NIDS and AIDS. The ACL and PDL parts of framework applies to all detection methodologies.

2. Deployment Environment: The computational and network context in which the IDS operates also shapes its latency profile, particularly regarding where inference occurs:

- *On-device detection* (e.g., on resource-constrained IoT devices) minimizes communication delay but may suffer from increased inference time due to limited processing capabilities.
- *Edge-based detection* offloads computation to nearby edge servers, reducing inference latency through higher performance hardware, though at the cost of added communication delay between the device and edge node.
- *Cloud-based detection* enables scalable, high-speed inference on powerful servers but introduces additional latency due to long-range communication over wide-area networks.
- *Federated IDS environments* further complicate latency profiles, as inference latency may be influenced by the synchronization cadence of model updates and the coordination overhead among distributed clients.

Latency framework

We define **latency** as the elapsed time between the arrival of the first malicious event at the monitored interface to its mitigation. Figure 1 provides a visual representation of this definition and all of the components of latency. It is also important to clarify the granularity at which latency is measured. As emphasized in Sect. 2.2, accurate latency calculation requires packet-level data with continuous timestamps, rather than aggregated flow-level records. Accordingly, our default definition treats latency on a per-packet basis: the elapsed time from the arrival of the first malicious packet to its mitigation. Ensemble algorithms such as XGBoost are used for per-packet

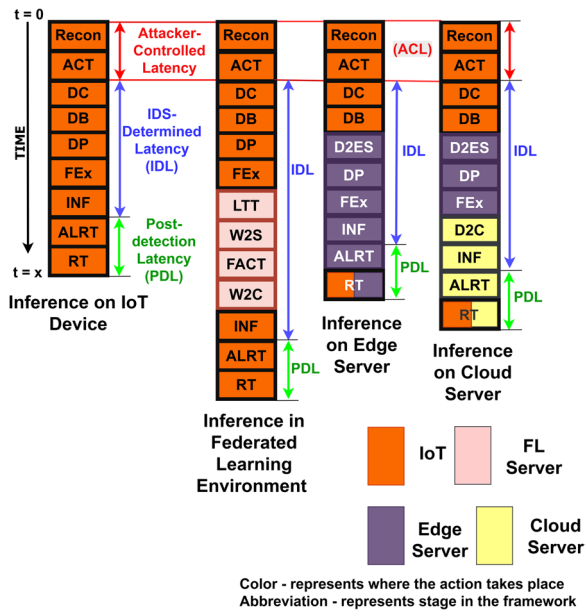


Fig. 1 Proposed latency framework. Refer to Table 4 for stage definition. When inference takes place on edge or cloud server, if an attack is detected, the server makes the response decision (e.g. shut down the system) and the IoT device executes it; the RT stage is therefore dual-colored to indicate this split

classification. XGBoost is employed in our ROSPaCe-based evaluation of the latency framework in Sect. 4.

However, deep learning models such as LSTM and other sequence-based architectures classify network traffic based on sequences of packets, not individual packets. In such cases, the system requires a series of packets (with fixed sequence length or variable sequence length) before producing a decision. Our framework accommodates this by treating the ADL as the interval from the arrival of the first malicious packet in the sequence until the model outputs its decision at the end of that sequence.

Scope and applicability of the latency framework

The proposed framework applies to both the offline (training) and real-time (inference) phases of IDS. During training, Attacker Controlled Latency (ACL), IDS-Determined Latency (IDL) elements such as Data Collection (DC), Data Buffering (DB), Data Processing (DP) and Feature Extraction (FEx) and Post Detection Latency (PDL) should be embedded within the dataset as features and determine how temporal continuity is represented. During inference, all of the above stages occur in real time as new packets arrive, allowing latency to be measured dynamically. This dual applicability ensures that the framework captures both static and runtime contributors to end-to-end delay.

Latency formalization

Now we introduce the technique to quantify latency. As illustrated in Fig. 1, latency can be decomposed into three distinct components:

- **Attacker-Controlled Latency (ACL):** Recon and ACT
- **IDS-Determined Latency (IDL):** DC, DB, DP, FEx, INF
- **Post-Detection Latency (PDL):** ALRT, RT

The combination of these three components is referred to as latency i.e. **end-to-end latency (e2e)**. Table 4 provides a detailed description of each term used in the latency equations.

$$ACL = t_{\text{Recon}} + t_{\text{ACT}} \quad (1)$$

$$IDL = t_{\text{DC}} + t_{\text{DB}} + t_{\text{DP}} + t_{\text{FEx}} + t_{\text{INF}} \quad (2)$$

$$PDL = t_{\text{ALRT}} + t_{\text{RT}} \quad (3)$$

$$L_{\text{e2e}} = ACL + IDL + PDL \quad (4)$$

Equations 1, 2, 3, and 4 define the baseline latency components in an environment-agnostic manner. For specific deployment environments, the expression for **IDL** changes depending on where inference is performed. Since latency is additive, the order of terms does not affect the result. In the subsequent formulations, the baseline **IDL** is substituted, and additional terms corresponding to that environment are included.

$$IDL^{(\text{Edge})} = IDL + t_{\text{D2ES}} \quad (5)$$

$$IDL^{(\text{Cloud})} = IDL + t_{\text{D2ES}} + t_{\text{D2C}} \quad (6)$$

$$IDL^{(\text{FL}^*)} = IDL^{(*)} + \mathbb{I}_{\text{wait}}(t_{\text{LTT}} + t_{\text{W2S}} + t_{\text{FACT}} + t_{\text{W2C}}), \quad \star \in \{\text{IoT}, \text{Edge}, \text{Cloud}\} \quad (7)$$

$$L_{\text{e2e}}^{(\text{env})} = ACL + IDL^{(\text{env})} + PDL \quad (8)$$

Case studies for latency framework

Figure 2 illustrates system-level latency decomposition using the proposed framework across IoT and Edge environments under two operating conditions: Nominal and Congested Network. For diagnostic emphasis, the ACL and PDL components are held constant, while the IDL stages vary to reveal the time effect of different stages.

In the **IoT** environment, all operations occur locally, and delays stem mainly from the computational stages

Table 4 Stages and corresponding notation in the latency framework in Fig. 1

Abbrev.	Notation	Definition
Recon	t_{Recon}	Reconnaissance time spent scanning or probing the target prior to attack initiation or payload delivery
ACT	t_{ACT}	Attacker communication time between attacker and target/victim during exploitation attempts
DC	t_{DC}	Data collection delay for capturing packets, flows, or logs from sensors or network taps
DB	t_{DB}	Data buffering or intermediate storage delay before processing
DP	t_{DP}	Data preprocessing delay (cleaning, parsing, normalization, or labeling)
FEx	t_{FEx}	Feature extraction time to compute discriminative features from processed data
INF	t_{INF}	Inference or classification time for producing a detection decision
ALRT	t_{ALRT}	Alert generation time for notifying a monitoring system or human operator
RT	t_{RT}	Response or mitigation initiation delay after an alert
D2ES	t_{D2ES}	Data transfer time from IoT device to edge server
D2C	t_{D2C}	Data or feature transfer time from edge to cloud for inference
LTT	t_{LTT}	Local training time on federated client
W2S	t_{W2S}	Client-to-server upload time for model weight updates
FACT	t_{FACT}	Federated aggregation compute time on central server
W2C	t_{W2C}	Server-to-client distribution time for aggregated model weights
–	$\mathbb{I}_{\text{wait}}$	Indicator: 1 if inference waits for new federated round, 0 otherwise
–	★	Environment placeholder where ★ ∈ {IoT, Edge, Cloud}

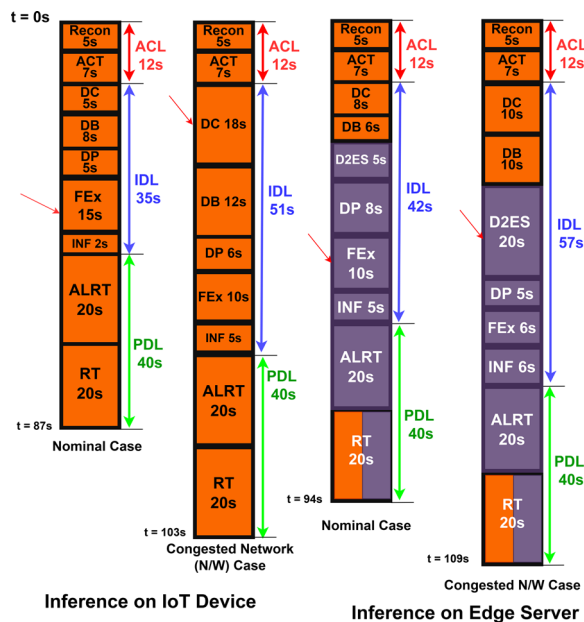


Fig. 2 Visualization of latency decomposition across IoT and Edge deployments under nominal and congested network conditions. The ACL and PDL components are held constant across environments to highlight how architectural choices affect latency optimization within the IDL component. Most prior studies reviewed in this work optimize isolated delays—such as communication, computation, or inference—rather than addressing end-to-end system latency. The proposed perspective emphasizes identifying the slowest process in the overall pipeline, indicated by the red arrow in each case

(DC, DB, DP, FEx) and inference (INF). The total IDL increases from 35 s in nominal case to 51 s in congested case, driven primarily by computation overhead rather than inference delay.

In the **Edge** environment, latency is dominated by data transfer (D2ES) and local computation. Under congestion, D2ES rises from 5 s to 20 s, markedly increasing total IDL, while inference remains nearly constant. This shows that communication between device and edge node dominates overall delay.

These cases demonstrate how the latency framework isolates the most time-dominant IDL stage, guiding system designers to focus on the bottleneck that most constrains responsiveness.

Attack detection latency (ADL)

ADL represents the operational realization of the proposed latency framework, quantifying the interval between attack onset and IDS detection. Figure 3 presents two framework case studies—one with inference executed on the IoT device and another within a Federated Learning environment. As established in Sect. 3.1, the start of an attack corresponds to the point at which malicious or suspicious content is first transmitted toward the target system. Reconnaissance does not necessarily mark the beginning of an attack, as it may be purely observational. Under this definition, the bounds of the proposed latency framework are as follows:

1. An attack may begin at any point within the Attacker-Controlled Latency (ACL) stage.

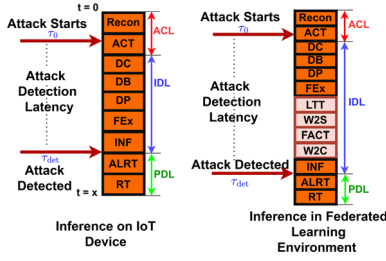


Fig. 3 Attack Detection Latency case studies. Refer to Table 4 for stage definition. Latency definition is per packet basis and therefore if attack is successfully detected then it will be at the end of inference stage

2. An attack can be detected only at the completion of the inference (INF) stage.

Notation. Let τ_0 denote the (asynchronous) *start time of the attack* (Sect. 3.1). Let τ_{det} be the *time the IDS issues a detection decision* (end of INF), and T_{obs} the end of the observation horizon. Detection happens at the end of inference stage because as mentioned in Sect. 3.1, the definition of latency is per packet basis.

$$ADL_{\text{det}} \triangleq \tau_{\text{det}} - \tau_0 \quad (\text{Attack Detection Latency}). \quad (9)$$

Case A: Detected during inference. If the system produces a detection decision for this attack within the observation horizon,

$$\mathbb{I}_{\text{hit}} = 1 \iff \exists \tau_{\text{det}} \leq T_{\text{obs}}, \quad (10)$$

then the measured ADL is

$$ADL^{(\text{hit})} = \tau_{\text{det}} - \tau_0, \quad \text{valid when } \mathbb{I}_{\text{hit}} = 1. \quad (11)$$

Case B: Not detected (missed) If no detection occurs by T_{obs} ,

$$\mathbb{I}_{\text{hit}} = 0 \iff \nexists \tau_{\text{det}} \leq T_{\text{obs}}, \quad (12)$$

latency will be greater than the observation horizon.

$$ADL^{(\text{miss})} = T_{\text{obs}} - \tau_0, \quad \text{when } \mathbb{I}_{\text{hit}} = 0. \quad (13)$$

This denotes that the true ADL satisfies

$$ADL > ADL^{(\text{miss})} \quad (14)$$

For ADL of a single packet, the attack may at any point within ACL stage. If detection occurs, it is registered at the end of the inference stage; therefore, PDL is excluded. Using Eq. 8 the resulting ADL is given by

$$\tau_{\text{det}} = ACL + IDL^{(\text{env})}, \quad (15)$$

Applying Eq. 11

$$ADL^{(\text{env})} = (ACL + IDL^{(\text{env})}) - \tau_0, \quad (16)$$

where τ_0 is the attack start time.

Reporting. Each attack instance is summarized by the pair

$$\left(ADL^{(\text{obs})}, \mathbb{I}_{\text{hit}} \right) \in \begin{cases} (\tau_{\text{det}} - \tau_0, 1), & \text{detected;} \\ (T_{\text{obs}} - \tau_0, 0), & \text{missed.} \end{cases}$$

Here $ADL^{(\text{obs})}$ could be either Case A, Eq. (11) or Case B, Eq. (13).

Demonstration using ROSPaCe dataset

The proposed latency framework is applied to compute attack detection latency (ADL) using a packet-level time-series dataset with interleaved benign and attack traffic. ROSPaCe is employed not to optimize detection performance, but to demonstrate that the framework enables empirical measurement of latency on real data, thereby validating its feasibility even in the absence of a fully latency-aware dataset. *The key distinction between this demonstration and prior studies reviewed in this work is that latency is not reported as aggregate inference time over the test set or as per-packet inference cost; instead, it is quantified as the dwell time of a malicious packet from its first appearance until detection in a system.*

ROSPaCe dataset published by Puccetti et al. (2024), is a large-scale, packet-based time-series dataset consisting of network and system-level traces collected during benign and attack scenarios. The full dataset has a total size of 38 GB. For computational efficiency, experiments are conducted using the published reduced dataset, `rospace_reduced` (11.9 GB). The reduced dataset comprises 15 CSV files. Each file encodes sequences of *observe* (benign) data points followed by *attack* data points. We define each observe–attack pair as a segment, with the benign and malicious portions denoted as sub-segments as visually depicted in Fig. 4. The dataset lacks explicit components of IDL; only Inference (INF) time can be measured in our experimentation. Since ROSPaCe does not provide reconnaissance stage for each category of attack, we assume $t_{\text{Recon}} = 0$ for each attack. Under this assumption, the timestamp associated with the first attack datapoint in an attack sub-segment is denoted as $t_{\text{ACT}}^{\text{start}}$, representing the onset of attacker–target communication during the exploitation attempt. Similarly, the timestamp corresponding to the final attack datapoint within the same sub-segment is denoted as $t_{\text{ACT}}^{\text{end}}$.

Consequently, applying Eq. 16 for ROSPaCe is computed as

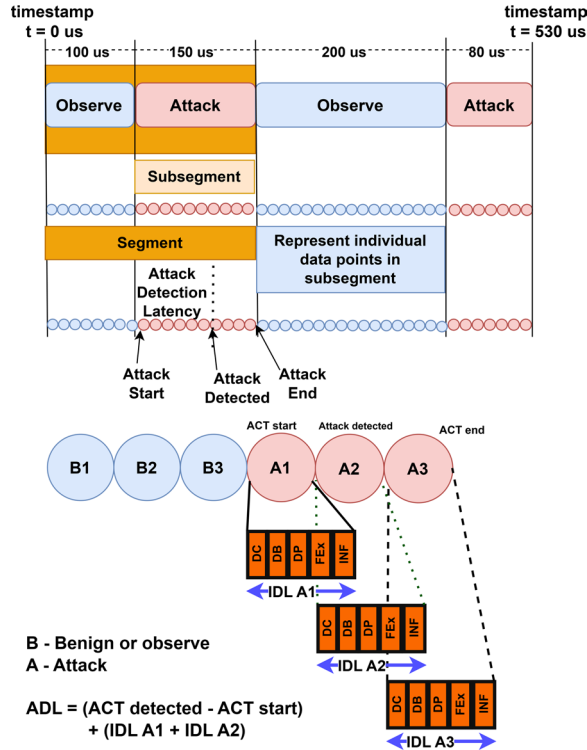


Fig. 4 This figure illustrates the definition of attack detection latency (ADL) and the segmentation of the ROSPaCe dataset into segments and sub-segments. Timestamps are recorded at microsecond resolution and occur at irregular intervals, reflecting non-uniform data logging consistent with the dataset. The figure conceptually maps IDS-determined latency (IDL) stages to individual datapoints to illustrate an idealized, fully latency-aware detection pipeline. However, ROSPaCe does not provide explicit annotations for Data Collection (DC), Data Buffering (DB), Data Processing (DP) and Feature Extraction (FEx) stages. As a result, only inference time can be measured during per packet binary classification using XGBoost

$$ADL = \left(t_{ACT}^{detected} - t_{ACT}^{start} \right) + \sum_{i=start}^{detected} INF_i, \quad (17)$$

where INF_i denotes the inference time associated with the i th datapoint. Average ADL (AADL) per attack category is formulated as follows: Let $\mathcal{K}$ denote the set of attack categories in the test set. For each category $k \in \mathcal{K}$, let $\mathcal{A}_k$ be the set of attack instances (attack sub-segments) of category k in the test set, with cardinality $N_k = |\mathcal{A}_k|$. The AADL for category k is defined as

$$\overline{ADL}_k = \frac{1}{N_k} \sum_{a \in \mathcal{A}_k} ADL(a), \quad (18)$$

where $ADL(a)$ is computed for each attack instance a using Eq. 17. In ROSPaCe, $\mathcal{K} = 6$ as shown in Table 5.

Table 5 Attack and observation sub-segments and data points in ROSPaCe

Type of attack	Sub-segments	Data points
nmap discovery	362	500,000
ros2 node crashing	48	2,124
ros2 reconnaissance	132	247,854
ros2 reflection	178	3,651
nmap SYN flood	17	5,634,833
metasploit SYN flood	282	11,658,671
Total attack	1019	18,047,133
Total observe	1014	4,883,605
Grand total	2033	22,930,738

Table 5 summarizes the six attack classes in ROSPaCe, their respective sub-segment counts, and data point volumes. Notably, the dataset is attack-majority, with approximately 78.7% of all data points belonging to attack sub-segments.

Feature engineering

Each data point contains 60 raw features derived from network packets, ROS2 messaging, and system resources. Features were selected for supervised learning using the following criteria:

1. Features constant across all CSV files were dropped.
2. Non-numeric features were excluded.
3. Features with $> 1\%$ missing values in the most populated file (csv0) were discarded.

After applying these rules, the feature set was reduced from 60 to 27 features.

Dataset splitting

To ensure temporal integrity, splitting was performed at the **segment level**. Segments were randomly partitioned into 80% training, 10% validation, and 10% test sets. Sub-segments and their data points inherited the label of the parent segment, ensuring no leakage between sets. The test set therefore consisted of 194 sub-segments (97 attack and 97 observe). No shuffling was applied at the data point level, since reordering individual time-series points would destroy temporal dependencies that are essential for latency measurement. It is worth noting, however, that in typical machine learning workflows shuffling is recommended at the sample level to prevent the model from memorizing sequential patterns or biases inherent in ordered data. In this study, shuffling was restricted to the *segment level* only, which balances the

need for temporal fidelity in latency experiments with the generalization benefits of randomized training partitions.

Computational environment

All experiments were executed on a system equipped with an Intel i9-12900HX processor (16 cores), 128 GB RAM, and an NVIDIA RTX 3000 GPU. The hardware configuration ensured that large-scale experiments on the `rospace_reduced` dataset (22.93 million data points after preprocessing) could be conducted without memory or compute bottlenecks.

Model selection

Among the candidate algorithms for supervised classification, XGBoost was selected as the primary model. Puccetti et al. (2024) compared XGBoost and Random Forest on the ROSPaCe dataset and reported that XGBoost consistently achieved higher classification accuracy and better scalability on large datasets. Furthermore, XGBoost is well-suited for tabular and heterogeneous features, which characterize the ROSPaCe dataset. Random Forest was therefore not pursued further after preliminary evaluation.

Hyperparameter tuning

For the large-scale data point classification task, XGBoost parameters were tuned using the Optuna framework, which employs Bayesian optimization to efficiently search hyperparameter space. Table 6 lists the tuned parameters and their corresponding values.

Regularization and overfitting control

Several regularization techniques were applied to prevent overfitting on the large dataset. These included L1 (`reg_alpha`) and L2 (`reg_lambda`) penalties, early stopping to terminate boosting when validation performance plateaued, and controlled subsampling of training data and features. These settings ensured that the trained models generalized well across experimental conditions.

Results

We perform per-packet binary classification . The test set comprises of 97 attack sub-segments and 97 observe sub-segments spanning six distinct attack categories. The test has 2,315,344 data points. At an overall accuracy of 93.49%, the AADL per-attack category, maximum ADL, and standard deviation are reported in Table 7.

Table 6 XGBoost Parameters Tuned Using Optuna

Parameter	Description	Value
<code>n_estimators</code>	Number of boosting rounds (trees)	231
<code>max_depth</code>	Maximum depth of a tree; controls complexity and overfitting risk	3
<code>learning_rate</code>	Step size shrinkage to balance convergence speed and stability	0.047
<code>subsample</code>	Fraction of training samples used to grow each tree	0.6002
<code>colsample_bytree</code>	Fraction of features sampled per tree	0.7978
<code>reg_alpha</code>	L1 regularization term; encourages sparsity	0.3383
<code>reg_lambda</code>	L2 regularization term; controls weight magnitudes	1.58e−06
<code>min_child_weight</code>	Minimum sum of instance weights in a child; prevents overfitting	2
<code>early_stopping_rounds</code>	Stops training if validation metric does not improve	62
<code>tree_method</code>	Training algorithm; GPU histogram-based tree growth	<code>gpu_hist</code>
<code>objective</code>	Binary classification with probabilistic outputs	<code>binary:logistic</code>
<code>base_score</code>	Initial prediction score for all instances	0.5

Table 7 Performance of XGBoost per attack category at an overall Accuracy: 93.49%. ADL is expressed in milli-seconds (ms)

Attack	Acc. (%)	Detected/total attacks	ADL (Avg/Max/Std) (ms)
<code>nmap_discovery</code>	73.21	42/42	762.820/9364.296/1927.873
<code>ros2_node_crashing</code>	0.00	0/7	N/A
<code>ros2_reconnaissance</code>	0.03	2/13	3190.579/5224.123/2033.545
<code>ros2_reflection</code>	0.00	0/15	N/A
<code>nmap_SYN_flood</code>	99.99	4/4	84.251/178.001/69.902
<code>metasploit_SYN_Flood</code>	99.38	16/16	0.000172/0.000172/0.000

Note that for the attack categories `ros2_node_crashing` and `ros2_reflection`, the classification accuracy is 0%, indicating that the model fails to correctly identify any of the 404 and 216 attack datapoints, respectively. This limitation is consistent with observations reported by Puccetti et al. (2024), who used ROSPaCe dataset and XGBoost model and stated “we show a very low capability in detecting the three ROS2 attacks. These attacks are less represented in the dataset, having shorter sequences with a smaller number of packets, because they lead the system to a failed state”. In these cases, the Attack Detection Latency (ADL) is not computed because it exceeds the observation horizon T_{obs} , as discussed in Eqs. 13 and 14. Consequently, both the average ADL (AADL) and the maximum ADL are reported as N/A in Table 7.

In contrast, for the `metasploit_SYN_flood` attack category, all 16 attacks in the test set are detected at the first datapoint of the attack sub-segment. As a result, both AADL and maximum ADL correspond to a very small value, equal to the inference time of a single datapoint i.e. 0.171873 micro-seconds.

The code and the feature-engineered ROSPaCe dataset used to generate the results reported in this table are publicly available at <https://github.com/SandyDash19/Intrusion-Detection-Latency-The-Neglected-Metric>.

Discussion

A standardized framework for measuring IDS latency is essential to enable meaningful comparison across intrusion detection studies. As observed in the literature review, many works report only partial latency metrics, such as inference time, communication delay, or computation time, while ignoring other stages of the detection pipeline. Such selective reporting makes it difficult to compare responsiveness of IDS designs. The framework proposed in this work addresses this issue by defining latency as the composite of ACL, IDL, and PDL, enabling a consistent end-to-end view of detection timeliness.

A key property of the proposed framework is its adaptability across deployment environments. The baseline formulation corresponds to on-device detection in an IoT environment, where all stages of the detection pipeline execute locally. As the deployment architecture evolves to include edge, cloud, or federated learning infrastructures, additional latency components naturally extend the baseline model. Because the framework is additive, these extensions allow the same latency formulation to scale smoothly across heterogeneous architectures while maintaining a consistent interpretation of ADL. This adaptability enables system designers to identify the dominant latency contributors in a given deployment and prioritize optimization efforts accordingly. For example,

in many practical systems, communication delays or data processing stages may dominate overall latency, making optimization of inference time alone insufficient.

Despite these advantages, several challenges arise when applying the proposed framework in practice. First, accurate computation of ADL requires datasets that satisfy specific temporal properties. In particular, datasets must provide packet-level observations with continuous timestamps and interleaved benign and attack traffic. Many commonly used IDS datasets do not meet these requirements, as they rely on aggregated flow records or lack precise temporal annotations. Second, comprehensive evaluation of IDL requires visibility into all stages of the detection pipeline, including DC, DB, DP, FEx, and, in distributed environments, communication and coordination stages such as D2ES, D2C, and federated learning synchronization operations. In real-world deployments, measuring these components reliably may require additional instrumentation and careful experimental design.

Another practical challenge concerns real-time measurement of the individual latency components. While the framework decomposes latency into well-defined stages, obtaining precise measurements for each stage across heterogeneous hardware platforms and network conditions may be non-trivial. Accurate latency profiling therefore requires standardized measurement methodologies and consistent reporting practices. Without such instrumentation, it may be difficult to reproduce latency measurements across different experimental settings or operational environments.

Future work will focus on validating the proposed framework using realistic measurements of IDL stages across multiple deployment environments, including IoT, edge, cloud, and federated IDS architectures. Such validation will require empirical characterization of computation, communication, and synchronization delays under realistic workloads. In addition, the development of latency-complete datasets with precise temporal annotations will be critical for enabling rigorous benchmarking of ADL. Together, these efforts will help establish latency-aware evaluation as a standard practice in IDS research.

Conclusion

This paper presents **latency** as a first-class metric for IDS, addressing its inconsistent treatment in prior IDS research. The proposed framework is necessary to formalize the temporal annotations of future datasets, enabling accurate and reproducible calculation of Attack Detection Latency (ADL). We formalize end-to-end latency as the composite of *Attacker-Controlled Latency* (ACL), *IDS-Determined Latency* (IDL), and *Post-Detection Latency* (PDL) within a unified framework spanning

IoT, edge, cloud, and federated environments. Using this framework, we empirically evaluate the ROSPaCe dataset to quantify ADL across multiple attack categories in realistic Cyber-Physical System (CPS) settings, demonstrating the framework's practical applicability. Achieving comprehensive latency-aware benchmarking requires temporally precise datasets, standardized reporting practices, and detection architectures explicitly optimized for rapid response. A key practical deployment challenge of the proposed framework lies in measuring each latency component in real time. Careful design of measurement mechanisms and reporting methodologies is therefore essential to ensure reliable and reproducible latency evaluation across heterogeneous deployment environments. Ultimately, an IDS that detects too late is indistinguishable from one that fails, underscoring that *timeliness* must be regarded as equally fundamental as accuracy in assessing IDS performance.

A Source list for Table 2

[A] Do not discuss latency (24 papers)

Berguiga et al. (2025), Misra et al. (2016), Mehta et al. (2025), Wu et al. (2023), Mahmoud (2023), Liu et al. (2025), Sahoo and Das (2025), Agnew et al. (2024), Libri et al. (2020), Mudgal et al. (2024), Marouane et al. (2023), Tian et al. (2020), Hattarki et al. (2021), Clavijo-Herrera et al. (2021), Aloul et al. (2021), Jamal et al. (2023), Christopher et al. (2021), Agarwal et al. (2022), Radhakrishna and Majd (2024), Li and Majd (2023), Tripathy et al. (2024), Mahadevappa et al. (2024), Ao et al. (2024) and Aldaej et al. (2024).

[B] Inference time (18 papers)

Al Ghamdi (2023), Musthafa et al. (2025), Zhang et al. (2022), Sayed et al. (2022), Okey et al. (2023), Dao and Lee (2024), Ricci et al. (2021), Zainudin et al. (2023), Govea et al. (2025), Gaikwad et al. (2025), Aman et al. (2022), Paolini et al. (2024), Wu et al. (2023), Borgioli et al. (2023), Xu et al. (2024), , Asulba et al. (2025), Yagiz et al. (2024) and Zhang et al. (2025).

[C] Compute time (4 papers)

Azhar et al. (2024), Janabi et al. (2022), Shan et al. (2022) and Wu et al. (2023).

[D] Communication time (29 papers)

Sakraoui et al. (2024), Anbalagan et al. (2023), Algarni et al. (2024), Zidi et al. (2024), Ghimire and Rawat (2022), Javed et al. (2023), Janabi et al. (2022), Chaganti (2025), Widanapathirana et al. (2014), Guo et al. (2022), Ahmed et al. (2023), Chang et al. (2023), Abdulkareem et al. (2019), Uddin and Kumar (2023), Aljebreen et al. (2023),

Nguyen et al. (2023), Khan et al. (2023), Moya Osorio et al. (2022), Srilakshmi et al. (2022), Aouedi et al. (2023), Adhikari and Hazra (2022), Liao et al. (2022), Zhu et al. (2022), De et al. (2022), Ramkumar et al. (2022), Ferrag et al. (2023), Hayat et al. (2024), Wang et al. (2023) and Abdulqadder and Zhou (2022).

[E] Inference + Compute time (3 papers)

Moudoud et al. (2022), Puccetti and Ceccarelli (2024) and Akem et al. (2024).

[F] Compute + Communication time (3 papers)

Yahyaoui et al. (2021), Syu et al. (2023) and Kumar et al. (2022).

[G] Compute + Communication + Inference time (5 papers)

Yang et al. (2022), Singh et al. (2021), Singh et al. (2022), Loukas et al. (2018) and Wang et al. (2022).

[H] Time overhead (5 papers)

Wu et al. (2016), Thankappan et al. (2024), Liang et al. (2020), Mohammadi et al. (2017) and Wu et al. (2023).

Abbreviations

ACL	Attacker-controlled latency
ADL	Attack detection latency
AADL	Average attack detection latency
CPS	Cyber-physical system
DC	Data collection
DB	Data buffering
DP	Data preprocessing
FEx	Feature extraction
IDL	IDS-determined latency
IDS	Intrusion detection system
INF	Inference
IoT	Internet of Things
PDL	Post-detection latency
ROSPaCe	Robot operating system 2 for smart passenger center dataset
ALRT	Alert generation time
RT	Response time
D2ES	Data transfer time from IoT device to edge server
D2C	Data transfer time from edge to cloud
LTT	Local training time in Federated Learning environment
W2S	Client-to-server upload time
FACT	Federated aggregation compute time
W2C	Server-to-client distribution time
ML	Machine learning
DL	Deep learning

Acknowledgements

The authors gratefully acknowledge Richard Atherton (atherton@pdx.edu) for his critical and constructive review of the manuscript, and for providing thoughtful and insightful feedback. Notes on Use of Large Language Models: ChatGPT was used only as editorial tools to assist with formatting and clarity. All conceptual contributions, methodology, analysis, interpretations, decisions, and responsibility for the final manuscript rest entirely with the authors.

Author Contributions

SD conceived the study, developed the latency framework, implemented the experiments, and drafted the manuscript. JMA contributed to conceptual development, critical revision of the manuscript and supervision. Both authors read and approved the final manuscript.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data Availability

The ROSPaCe dataset used in this study is publicly available from the original authors (Puccetti et al. 2024).

Code Availability

Code and feature-engineered ROSPaCe data used for generating the experimental results in Table 7 are available at <https://github.com/SandyDash19/Intrusion-Detection-Latency-The-Neglected-Metric>.

Declarations

Conflict of interest

The authors declare that they have no conflict of interest.

Received: 24 November 2025 Accepted: 11 March 2026

Published online: 25 March 2026

References

- Abdulkareem KH, Mohammed MA, Gunasekaran SS et al (2019) A review of fog computing and machine learning: concepts, applications, challenges, and open issues. *IEEE Access* 7:153123–153140
- Abdulqadder IH, Zhou S (2022) SliceBlock: context-aware authentication handover and secure network slicing using DAG-BLOCKCHAIN IN EDGE-ASSISTED SDN/NFV-6G environment. *IEEE Internet Things J* 9(18):18079–18097
- Adhikari M, Hazra A (2022) 6G-enabled ultra-reliable low-latency communication in edge networks. *IEEE Commun Stand Mag* 6(1):67–74
- Agarwal N, Quadir MA, Vigneswaran T, Punitha K, Sivaraman AK (2022) A robust pipeline approach for DDoS classification using machine learning. In: *International conference on intelligent computing, instrumentation and control technologies (ICICICT)*. Kannur, India, pp 1621–1627
- Agnew D, Rice-Bladykas A, McNair J (2024) Detection of zero-day attacks in a software-defined LEO constellation network using enhanced network metric predictions. *IEEE Open J Commun Soc* 5:6611–6634
- Ahmed I, Anisetti M, Ahmad A, Jeon G (2023) A multilayer deep learning approach for Malware classification in 5G-enabled IIoT. *IEEE Trans Ind Inf* 19(2):1495–1503
- Akem ATJ, Gucciardo M, Fiore M (2024) Ultra-low latency user-plane cyberattack detection in SDN-based smart grids. In: *15th ACM international conference on future energy system (e-Energy '24)*. Singapore, Singapore, pp 676–682
- Al Ghamdi MA (2023) A fine-grained system driven of attacks over several new representation techniques using machine learning. *IEEE Access* 11:96615–96625
- Aldaej A et al (2024) Ensemble technique of intrusion detection for IoT-edge platform. *Sci Rep* 14:11703
- Algarni A, Acarer T, Ahmad Z (2024) An edge computing-based preventive framework with machine learning integration for anomaly detection and risk management in maritime wireless communications. *IEEE Access* 12:53646–53663
- Aljebreen M, Alrayes FS, Maray M, Aljameel SS, Salama AS, Motwakel A (2023) Modified equilibrium optimization algorithm with deep learning-based DDoS attack classification in 5G networks. *IEEE Access* 11:108561–108570
- Aloul F, Zualkernan I, Abdalgawad N, Hussain L, Sakhrini D (2021) Network intrusion detection on the IoT edge using adversarial autoencoders. In: *International conference on information technology (ICIT)*. Amman, Jordan, pp 120–125
- Aman MN, Basheer H, Wong JW, Xu J, Lim HW, Sikdar B (2022) Machine-learning-based attestation for the internet of things using memory traces. *IEEE Internet Things J* 9(20):20431–20443
- Anbalagan S, Raja G, Gurumoorthy S, Suresh RD, Ayyakannu K (2023) Blockchain assisted hybrid intrusion detection system in autonomous vehicles for industry 5.0. *IEEE Trans Consum Electron* 69(4):881–889
- Ao Y, Shi H, Wang N et al (2024) An accurate and lightweight intrusion detection model deployed on edge network devices. In: *International joint conference on neural networks (IJCNN)*. Yokohama, Japan, pp 1–8
- Aouedi O, Piamrat K, Muller G, Singh K (2023) Federated semisupervised learning for attack detection in industrial internet of things. *IEEE Trans Ind Inf* 19(1):286–295
- Asulba B, Souto PF, Almeida L (2025) Bringing IoT intrusion detection to the edge. In: *8th international conference on future networks and distributed systems (ICFNDS'24)*. NY, USA, New York, pp 295–304
- Azhar M, Perveen S, Iqbal A, Lee B (2024) IDRandom-forest: advanced random forest for real-time intrusion detection. *IEEE Access* 12:113842–113854
- Berguiga A, Harchay A, Massaoudi A (2025) HIDS-IoMT: a deep learning-based intelligent intrusion detection system for the internet of medical things. *IEEE Access* 13:32863–32882
- Borgioli N, Phan LTX, Aromolo F, Biondi A, Buttazzo G (2023) Real-time packet-based intrusion detection on edge devices. In: *Cyber-physical systems and internet of things week (CPS-IoT Week'23)*. San Antonio, TX, USA, pp 234–240
- Chaganti KC (2025) A scalable, lightweight AI-driven security framework for IoT ecosystems: optimization and game theory approaches. *IEEE Access* 13:72235–72247
- Chang H, Kodialam M, Lakshman TV, Mukherjee S, Van der Merwe J, Zaheer Z (2023) MAGNet: machine learning guided application-aware networking for data centers. *IEEE Trans Cloud Comput* 11(1):291–307
- Christopher V, Aathman T, Mahendrakumaran K et al (2021) Minority resampling boosted unsupervised learning with hyperdimensional computing for threat detection at the edge of internet of things. *IEEE Access* 9:126646–126657
- Clavijo-Herrera M, Banda-Almeida J, Iza C (2021) Performance evaluation in Misbehavior detection techniques for DoS attacks in VANETs. In: *ACM symposium on performance evaluation of wireless ad hoc, sensor, and ubiquitous networks (PE-WASUN)*. Alicante, Spain, pp 73–80
- Dao TN, Lee H (2024) COOL: conservation of output links for pruning algorithms in network intrusion detection. *IEEE Internet Things J* 11(5):8906–8920
- De S, Bermudez-Edo M, Xu H, Cai Z (2022) Deep generative models in the industrial internet of things: a survey. *IEEE Trans Ind Inf* 18(9):5728–5737
- Ferrag MA, Friha O, Kantarci B et al (2023) Edge learning for 6G-enabled internet of things: a comprehensive survey of vulnerabilities, datasets, and defenses. *IEEE Commun Surv Tutor* 25(4):2654–2713
- Gaikwad NB, Khare SK, Mendhe D, Mir H, Kosta S, Acharya UR (2025) FPGA SoC implementation of adaptive deep neural network-based multimodal edge intelligence for internet of medical things. *IEEE Access* 13:134041–134056
- Ghimire B, Rawat DB (2022) Recent advances on federated learning for cybersecurity and cybersecurity for federated learning for internet of things. *IEEE Internet Things J* 9(11):8229–8249
- Govea J, Gutierrez R, Villegas-Ch W, Maldonado NA (2025) Hybrid AI for predictive cyber risk assessment: federated graph-transformer architecture with explainability. *IEEE Access* 13:122187–122206
- Guo K, Chen J, Dong P, Liu S, Gao D (2022) FullSight: a feasible intelligent and collaborative framework for service function chains failure detection. *IEEE Trans Netw Serv Manag* 19(4):4546–4565
- Hattarki R, Houji S, Dhage M (2021) Real-time intrusion detection system for IoT networks. In: *International conference on convergence technology (I2CT)*. Maharashtra, India, pp 1–5
- Hayat RF, Aurangzeb S, Aleem M, Srivastava G, Lin JCW (2024) ML-DDoS: a blockchain-based multilevel DDoS mitigation mechanism for IoT environments. *IEEE Trans Eng Manage* 71:12605–12618
- Hutchins EM, Cloppert MJ, Amin RM (2011) Intelligence-driven computer network defense informed by analysis of adversary Campaigns and intrusion

- kill chains. In: 6th international conference on information warfare and security (ICIWS), pp 113–125
- Jamal MH et al (2023) A comparison of re-sampling techniques for detection of multi-step attacks on deep learning models. *IEEE Access* 11:127446–127457
- Janabi AH, Kanakis T, Johnson M (2022) Overhead reduction technique for software-defined network-based intrusion detection systems. *IEEE Access* 10:66481–66491
- Janabi AH, Kanakis T, Johnson M (2022) Convolutional neural network based algorithm for early warning proactive system security in software defined networks. *IEEE Access* 10:14301–14310
- Javed Y, Khayat MA, Elghariani AA, Ghafoor A (2023) PRISM: a hierarchical intrusion detection architecture for large-scale cyber networks. *IEEE Trans Depend Secure Comput* 20(6):5070–5086
- Khan MA, Kumar N, Mohsan SAH, Khan WU, Nasralla MM, Alsharif M (2023) Swarm of UAVs for network management in 6G: a technical review. *IEEE Trans Netw Serv Manag* 20(1):741–761
- Kumar P, Kumar R, Gupta GP, Tripathi R (2022) BDEdge: blockchain and deep-learning for secure edge-envisioned green CAVs. *IEEE Trans Green Commun Netw* 6(3):1330–1339
- Liang J, Lin Q, Chen J, Zhu Y (2020) A filter model based on hidden generalized mixture transition distribution model for intrusion detection system in vehicle ad hoc networks. *IEEE Trans Intell Transp Syst* 21(7):2707–2722
- Liao Z, Pang X, Zhang J, Xiong B, Wang J (2022) Blockchain on security and forensics management in edge computing for IoT: a comprehensive survey. *IEEE Trans Netw Serv Manag* 19(2):1159–1175
- Libri A, Bartolini A, Benini L (2020) pAella: edge AI-based real-time Malware detection in data centers. *IEEE Internet Things J* 7(10):9589–9599
- Li D, Majd NE (2023) Intrusion detection in IoT leveraged by multiaccess edge computing using machine learning. In: IEEE international performance, computing, and communications conference (IPCCC). Anaheim, CA, USA, pp 441–446
- Lippmann RP, Fried DJ, Graf I, Haines JW, Kendall KR, McClung D et al (2000) Evaluating intrusion detection systems: the 1998 DARPA off-line intrusion detection evaluation. In: DARPA information survivability conference and exposition (DISCEX), vol 2. Hilton Head, SC, USA, pp 12–26
- Liu J, Wang G, Zong X, Ning B, He K (2025) EfficientTransformer: a dynamic anomaly detection model for industrial control networks. *IEEE Access* 13:37931–37945
- Loukas G, Vuong T, Heartfield R, Sakellari G, Yoon Y, Gan D (2018) Cloud-based cyber-physical intrusion detection for vehicles using deep learning. *IEEE Access* 6:3491–3508
- Mahadevappa P et al (2024) A secure edge computing model using machine learning and IDS to detect and isolate intruders. *MethodsX* 12:102597
- Mahmoud SS (2023) Practical aspects of perimeter intrusion detection and nuisance suppression for distributed fiber-optic sensors. *IEEE Trans Instrum Meas* 72:1–11
- Mandiant. M-Trends 2020: insights into today's breach detection and response. FireEye Mandiant; 2020. Available from: <https://www.mandiant.com/resources/reports/m-trends-2020>
- Marouane H, Dandoush A, Amour L, Erbad A (2023) Performance evaluation of machine learning-based misbehavior detection systems in VANETs: a comprehensive study. In: International symposium on networks, computers and communications (ISNCC). Doha, Qatar, pp 1–6
- Mehta N, Bharot N, Breslin JG, Verma P (2025) PPFL-DCS: privacy-preserving federated learning using neural transformer and leveraging dynamic client selection to accommodate data diversity. *IEEE Access* 13:94225–94238
- Misra S, Goswami S, Taneja C (2016) Multivariate data fusion-based learning of video content and service distribution for cyber physical social systems. *IEEE Trans Comput Soc Syst* 3(1):1–12
- MITRE Corporation (2025) MITRE ATT&CK® knowledge base. Available from: <https://attack.mitre.org/>
- Mohammadi R, Javidan R, Conti M (2017) SLICOTS: an SDN-based lightweight countermeasure for TCP SYN flooding attacks. *IEEE Trans Netw Serv Manag* 14(2):487–497
- Moudoud H, Mlika Z, Khoukhi L, Cherkaoui S (2022) Detection and prediction of FDI attacks in IoT systems via hidden Markov model. *IEEE Trans Netw Sci Eng* 9(5):2978–2990
- Moya Osorio DP, Ahmad I, Sánchez JDV, Gurtov A, Schollers J, Kutila M (2022) Towards 6G-enabled internet of vehicles: security and privacy. *IEEE Open J Commun Soc* 3:82–105
- Mudgal A, Verma A, Singh M, Sahoo KS, Elmroth E, Bhuyan M (2024) FloRa: flow table low-rate overflow reconnaissance and detection in SDN. *IEEE Trans Netw Serv Manag* 21(6):6670–6683
- Musthafa MB, Huda S, Nguyen TT, Kodera Y, Nogami Y (2025) Optimized ensemble deep learning for real-time intrusion detection on resource-constrained raspberry Pi devices. *IEEE Access* 13:113544–113556
- Nguyen DDN, Sood K, Xiang Y, Gao L, Chi L, Yu S (2023) Toward IoT node authentication mechanism in next generation networks. *IEEE Internet Things J* 10(15):13333–13341
- Okey OD, Melgarejo DC, Saadi M, Rosa RL, Kleinschmidt JH, Rodríguez DZ (2023) Transfer learning approach to IDS on cloud IoT devices using optimized CNN. *IEEE Access* 11:1023–1038
- Paolini E, de Marinis L, Scano D, Paolucci F (2024) In-line any-depth deep neural networks using P4 switches. *IEEE Open J Commun Soc* 5:3556–3567
- Puccetti T, Ceccarelli A (2024) Detection latencies of anomaly detectors: an overlooked perspective. In: IEEE international symposium on software reliability engineering (ISSRE), pp 304–310
- Puccetti T, Ceccarelli A, Catricalà M, Marchetti M, Pappalardo A, Vecchio M et al (2024) ROSPaCe: intrusion detection dataset for a ROS2-based cyber-physical system and IoT networks. *Sci Data* 11(311)
- Radhakrishna T, Majd NE (2024) Edge computing ransomware detection in IoT using machine learning. In: International conference on computing, networking and communications (ICNC). Big Island, HI, USA, pp 244–248
- Ramkumar G, Prabu AGRT, Nirmala P, Ramya G (2022) Strong and stable data communication using artificial intelligence method in mobile Ad-Hoc networks. In: International conference on innovative computing, intelligent communication and smart electrical systems (ICSES), pp 1–10
- Ricci M, Štitić B, Urbinati L, De Guglielmo G, Vasquez JAT, Carloni LP et al (2021) Machine-learning-based microwave sensing: a case study for the food industry. *IEEE J Emerging Sel Topics Circuits Syst* 11(3):503–514
- Sahoo SS, Das D (2025) Design of key agreement authentication scheme with IDS using OS-RVFL neural network for secure fog computing. *IEEE Internet Things J* 12(14):28521–28530
- Sakraoui S, Ahmim A, Derdour M, Ahmim M, Namane S, Ben DI (2024) FBMP-IDS: FL-based blockchain-powered lightweight MPC-secured IDS for 6G networks. *IEEE Access* 12:105887–105905
- Sayed MSE, Le-Khac NA, Azer MA, Jurcut AD (2022) A flow-based anomaly detection approach with feature selection method against DDoS attacks in SDNs. *IEEE Trans Cognit Commun Netw* 8(4):1862–1880
- Shan G, Zhao B, Clavin JR, Zhang H, Duan S (2022) Poligraph: intrusion-tolerant and distributed fake news detection system. *IEEE Trans Inf Forens Secur* 17:28–41
- Singh P, Kaur A, Aujla GS, Batth RS, Kanhere S (2021) DaaS: dew computing as a service for intelligent intrusion detection in edge-of-things ecosystem. *IEEE Internet Things J* 8(16):12569–12577
- Singh S, Sulthana R, Shewale T, Chamola V, Benslimane A, Sikdar B (2022) Machine-learning-assisted security and privacy provisioning for edge computing: a survey. *IEEE Internet Things J* 9(1):236–260
- Sommer R, Paxson V (2010) Outside the closed world: on using machine learning for network intrusion detection. In: IEEE symposium on security and privacy. Oakland, CA, USA, pp 305–316
- Srilakshmi U, Alghamdi SA, Vuyyuru VA, Veeraiah N, Alotaibi Y (2022) A secure optimization routing algorithm for mobile ad hoc networks. *IEEE Access* 10:14260–14269
- Syu JH, Lin JCW, Srivastava G, Yu K (2023) A comprehensive survey on artificial intelligence empowered edge computing on consumer electronics. *IEEE Trans Consum Electron* 69(4):1023–1034
- Thankappan M, Rifa-Pous H, Garrigues C (2024) A signature-based wireless intrusion detection system framework for multi-channel man-in-the-middle attacks against protected Wi-Fi networks. *IEEE Access* 12:23096–23121
- Tian Z, Luo C, Qiu J, Du X, Guizani M (2020) A distributed deep learning system for web attack detection on edge devices. *IEEE Trans Ind Inf* 16(3):1963–1971
- Tripathy AK, Mishra AK, Panda R (2024) Performance comparison and analysis of machine learning and deep learning models for network intrusion detection in IoT-edge frameworks. In: International conference on emerging trends in information technology and engineering (ICETITE). Vellore, India, pp 1–6

- Uddin R, Kumar SAP (2023) SDN-based federated learning approach for satellite-IoT framework to enhance data security and privacy in space communication. *IEEE J Radio Frequency Identif* 7:424–440
- Usman Y, Upadhyay A, Gyawali P, Chataut R (2024) Is generative AI the next tactical cyber weapon for threat actors? A deep dive into GenAI-enabled offensive capabilities. *arXiv preprint arXiv:2408.12806*
- Wang Y, Yang S, Ren X, Zhao P, Zhao C, Yang X (2022) IndustEdge: a time-sensitive networking enabled edge-cloud collaborative intelligent platform for smart industry. *IEEE Trans Ind Inf* 18(4):2386–2398
- Wang C, Yuan Z, Zhou P, Xu Z, Li R, Wu DO (2023) The security and privacy of mobile-edge computing: an artificial intelligence perspective. *IEEE Internet Things J* 10(24):22008–22032
- Widanapathirana C, Li JC, Ivanovich MV, Fitzpatrick PG (2014) Adaptive statistical signatures of network soft-failures in user devices. *Comput J* 57(8):1262–1278
- Wu Y et al (2023) Paradise: real-time, generalized, and distributed provenance-based intrusion detection. *IEEE Trans Depend Secure Comput* 20(2):1624–1640
- Wu J, Ota K, Dong M, Li C (2016) A hierarchical security framework for defending against sophisticated attacks on wireless sensor networks in smart cities. *IEEE Access* 4:416–424
- Wu Y, Nie L, Wang S, Ning Z, Li S (2023) Intelligent intrusion detection for internet of things security: a deep convolutional generative adversarial network-enabled approach. *IEEE Internet Things J* 10(4):3094–3106
- Wu Y, Nie L, Wang S, Ning Z, Li S (2023) Intelligent intrusion detection for internet of things security: a deep convolutional generative adversarial network-enabled approach. *IEEE Internet Things J* 10(4):3094–3106
- Wu G, Wang H, Zhang H, Zhao Y, Yu S, Shen S (2023) Computation offloading method using stochastic games for software-defined-network-based multiagent mobile edge computing. *IEEE Internet Things J* 10(20):17620–17634
- Xu H, Wu D, Lu Y, Lu J, Zeng H (2024) Models on the move: towards feasible embedded AI for intrusion detection on vehicular can bus. In: *USENIX annual technical conference (USENIX ATC 24)*. Santa Clara CA USA, pp 1049–1063
- Yagiz MA, MohajerAnsari P, Pesé MD, Goktas P (2024) Transforming in-vehicle network intrusion detection: VAE-based knowledge distillation meets explainable AI. In: *6th workshop on CPS&IoT security and privacy (CPSIoT-Sec'24)*. Salt Lake City, UT, USA, pp 93–103
- Yahyaoui A, Abdellatif T, Yangui S, Attia R (2021) READ-IoT: reliable event and anomaly detection framework for the internet of things. *IEEE Access* 9:24168–24186
- Yang L, Song Y, Gao S, Hu A, Xiao B (2022) Griffin: real-time network intrusion detection system via ensemble of autoencoder in SDN. *IEEE Trans Netw Serv Manag* 19(3):2269–2281
- Zainudin A, Ahakonye LAC, Akter R, Kim DS, Lee JM (2023) An efficient hybrid-DNN for DDoS detection and classification in software-defined IIoT networks. *IEEE Internet Things J* 10(10):8491–8504
- Zhang L, Yan X, Ma D (2022) A binarized neural network approach to accelerate in-vehicle network intrusion detection. *IEEE Access* 10:123505–123520
- Zhang B, Wang Z, Miao W (2025) Real-time intrusion detection method based on frequency domain simplified dual-PINN. In: *8th international conference on electronic information technology and computer engineering (EITCE'24)*. NY, USA, New York, pp 292–299
- Zhu L, Liang H, Wang H, Ning B, Tang T (2022) Joint security and train control design in blockchain-empowered CBTC system. *IEEE Internet Things J* 9(11):8119–8129
- Zidi S, Alaya B, Moulahi T, Al-Shargabi A, Khediri SE (2024) Fault prediction and recovery using machine learning techniques and the HTM algorithm in vehicular network environment. *IEEE Open J Intell Transp Syst* 5:132–145

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.